

1. Write program demonstrates how to use regular expressions in Python to match and search for patterns in text.

Aim

To write a Python program that demonstrates the use of regular expressions to match and search for patterns in text.

Algorithm

1. Start the program.
2. Import the re module.
3. Store a sample text in a variable.
4. Define a regular expression pattern for an email address.
5. Use re.search() to find the email in the text.
6. Define a regular expression pattern for a phone number.
7. Use re.search() to find the phone number.
8. Display the matched email and phone number.
9. Stop the program.

Python Code

```
import re
text = "My email is student123@gmail.com and my phone number is 9876543210."
email_pattern = r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b'
email_match = re.search(email_pattern, text)
if email_match:
    print("Email found:", email_match.group())
phone_pattern = r'\b\d{10}\b'
phone_match = re.search(phone_pattern, text)
if phone_match:
    print("Phone number found:", phone_match.group())
word_pattern = r'\w*\d+\w*'
words = re.findall(word_pattern, text)
print("Words containing digits:", words)
```

Output

```
.. Email found: student123@gmail.com
   Phone number found: 9876543210
   Words containing digits: ['student123', '9876543210']
```

Result

Thus, the Python program to demonstrate the use of regular expressions for matching and searching patterns in text was successfully executed and the output was verified.

2.Implement a basic finite state automaton that recognizes a specific language or pattern. In this example, we'll create a simple automaton to match strings ending with 'ab' using python.

Aim

To implement a basic Finite State Automaton (FSA) in Python that recognizes strings ending with "ab".

Algorithm

1. Start the program.
2. Define the states of the automaton.
3. Read the input string.
4. Process each character one by one and change the state.
5. After processing all characters, check the final state.
6. If the final state is the accepting state, display "Accepted".
7. Otherwise, display "Rejected".
8. Stop the program.

Python Code

```
def fsa_ends_with_ab(string):  
    state = 0  
  
    for ch in string:  
        if state == 0:  
            if ch == 'a':  
                state = 1  
            else:  
                state = 0  
  
        elif state == 1:  
            if ch == 'b':  
                state = 2  
            elif ch == 'a':  
                state = 1  
            else:  
                state = 0  
  
        elif state == 2:  
            if ch == 'a':  
                state = 1  
            else:  
                state = 0  
  
    if state == 2:  
        print("Accepted")  
    else:  
        print("Rejected")
```

```
text = input("Enter a string: ")  
fsa_ends_with_ab(text)
```

```
Enter a string: cab  
Accepted
```

```
• Enter a string: baaa  
Rejected
```

Result

Thus, the Python program to implement a Finite State Automaton (FSA) that recognizes strings ending with "ab" was successfully executed and the output was verified.

3. Write program demonstrates how to perform morphological analysis using the NLTK library in Python.

Aim

To write a Python program that demonstrates morphological analysis using the **NLTK** library by performing stemming and lemmatization.

Algorithm

1. Start the program.
2. Import the required NLTK modules.
3. Create a list of sample words.
4. Initialize the **Porter Stemmer** and **WordNet Lemmatizer**.
5. Apply stemming to each word.
6. Apply lemmatization to each word.
7. Display the original word, stemmed word, and lemmatized word.
8. Stop the program.

Python Code

```
import nltk

from nltk.stem import PorterStemmer, WordNetLemmatizer

nltk.download('wordnet')

words = ["running", "played", "better", "cars", "studies"]

stemmer = PorterStemmer()

lemmatizer = WordNetLemmatizer()

print("Word\t\tStem\t\tLemma")

for word in words:

    stem = stemmer.stem(word)

    lemma = lemmatizer.lemmatize(word)

    print(word, "\t", stem, "\t\t", lemma)
```

Sample Output

```
... Word          Stem          Lemma
    running        run          running
    played   play    played
    better   better
    cars     car     car
    studies      studi    study
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

Result

Thus, the Python program to perform **morphological analysis** using the **NLTK** library was successfully executed using **stemming** and **lemmatization**, and the output was verified